

深圳大学实验报告

课程名称：人工智能

实验项目名称：基于 YOLOv11 的行人和车辆检测

学 院：电子与信息工程学院

专 业：电子信息工程

指导教师：戴林蕙

报告人：侯泓宇

学号：2024280180

班级：电信 2401

实验时间：2026 年 3 月 23 日

实验报告提交时间：2026 年 4 月 22 日

教 务 部 制

一、实验目的与要求：

- 了解 YOLO 系列目标检测算法的发展历程，掌握 YOLOv11 的网络结构与核心改进。
- 学习在 Anaconda 中创建独立虚拟环境，并完成 Ultralytics 框架的安装与配置。
- 掌握通过 Roboflow 获取小规模标注数据集的方法，无需手工标注即可完成数据准备。
- 能够在 CPU 环境下对 YOLOv11n 进行轻量级微调训练，理解训练流程与超参数含义。
- 学会使用训练好的权重对图片和视频进行推理，解读检测结果与评估指标。

二、实验方法与步骤：

1. 创建并激活虚拟环境

打开 Anaconda Prompt，依次执行：

```
1 conda create -n yolo11 python=3.10 -y
2 conda activate yolo11
```

2. 安装 Ultralytics

```
1 pip install ultralytics
```

安装完成后验证：

```
1 python -c "from ultralytics import YOLO; print('安装成功')"
```

3. 从 Roboflow 下载数据集

本实验使用 Roboflow 上的公开中小型数据集（约 4584 张，含行人与车辆标注），无需 GPU，下载后即可直接用于训练。步骤如下：

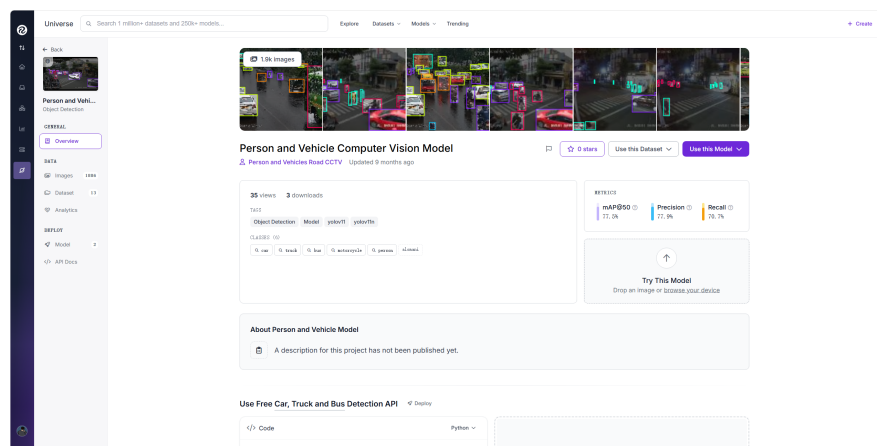


图 1 Roboflow 下载数据集

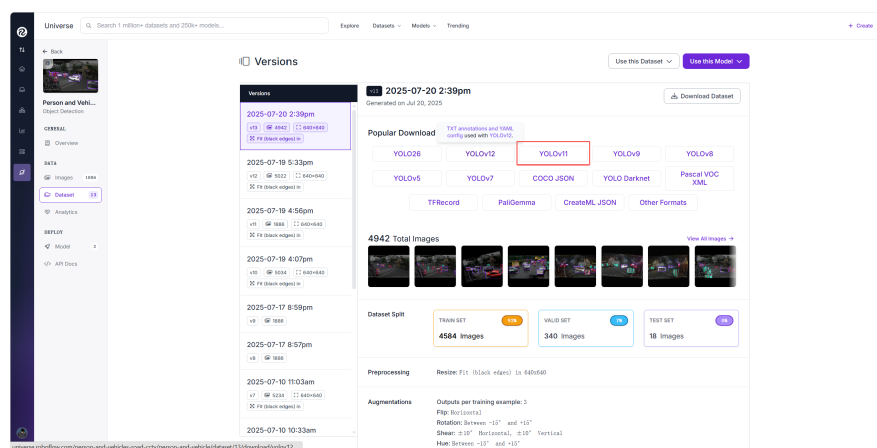


图 2 YOLOv11 格式

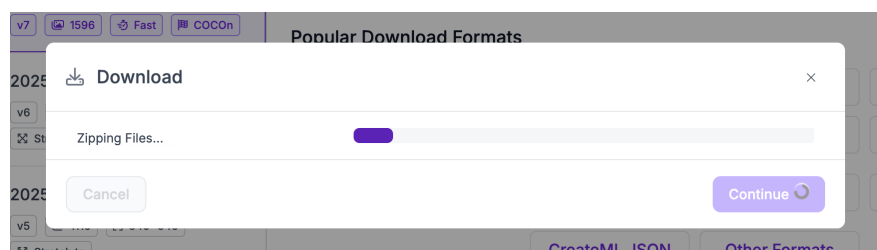


图 3 下载好啦

- 访问 <https://universe.roboflow.com/search?q=vehicle+pedestrian+detection>，注册免费账号并登录。
- 在搜索框中搜索 **”vehicle pedestrian detection”**，选择一个 YOLO 格式 (YOLOv11) 的公开数据集，点击 Download。

c. 格式选择 **YOLOv11**，下载方式选择 **show download code**，复制生成的 Python 代码片段。

d. 安装 Roboflow SDK 并运行下载代码：

```
1 pip install roboflow
```

e. 将复制的代码粘贴到 `download_data.py` 并运行，数据集会自动下载到本地并生成 `data.yaml`。

 test	2026/4/13 16:54	文件夹
 train	2026/4/13 17:18	文件夹
 valid	2026/4/13 17:18	文件夹
 data.yaml	2026/4/13 16:54	Yaml 源文件
 README.dataset.txt	2026/4/13 16:54	文本文档
 README.roboflow.txt	2026/4/13 16:54	文本文档

图 4 下载好的数据集目录结构

4. 确认数据集目录结构

下载完成后目录结构如下（路径以实际下载位置为准）：

```
1 dataset/  
2   train/  
3     images/    # 训练图片  
4     labels/    # 对应 YOLO 格式标注 (.txt)  
5   valid/  
6     images/    # 验证图片  
7     labels/  
8   data.yaml    # 自动生成，包含路径和类别信息
```

5. 编写并运行训练脚本

新建 `train.py`，内容见第三节，然后运行：

```
1 python train.py
```

注意：无 GPU 的电脑训练 10 个 epoch 约需 30–60 分钟，请耐心等待。

6. 推理验证

训练完成后运行推理脚本：

```
1 python infer.py
```

检测结果图片保存在 runs/detect/ 目录下。

三、实验过程及内容：

(1) 训练脚本 train.py

```
1 from ultralytics import YOLO
2
3 #加载YOLOv11 nano预训练权重（首次运行自动下载，约5MB）
4 model = YOLO('yolo11n.pt')
5
6 #注意：data填写data.yaml的完整路径
7 #如果train.py和数据集文件夹放在同一目录，可以用下面的相对路径：
8 results = model.train(
9     data='data.yaml',
10     epochs=10, # CPU训练建议10轮，约30-60分钟
11     imgsz=416, #降低分辨率以加快CPU训练速度
12     batch=4, # CPU训练batch设小一点
13     device='cpu', #无GPU则使用CPU
14     workers=2,
15     project='runs/detect',
16     name='yolo11_vp',
17     val=True,
18 )
19
20 print("训练完成！权重保存于:", results.save_dir)
```

(2) 推理脚本 infer.py

```
1 from ultralytics import YOLO
2
3 # 加载训练好的权重
4 model = YOLO('runs/detect/yolo11_vp/weights/best.pt')
5
6 # 对图片进行推理（将 test.jpg 替换为你自己的图片）
7 results = model.predict(
8     source='test1.jpg',
```

```
9     conf=0.25,          # 置信度阈值，低于此值的检测框会被过滤
10     iou=0.45,          # NMS 去重阈值
11     save=True,         # 保存标注结果图
12     device='cpu',
13 )
14
15 # 打印检测到的目标
16 for r in results:
17     for box in r.bboxes:
18         cls = int(box.cls)
19         conf = float(box.conf)
20         name = model.names[cls]
21         print(f"检测到: {name}, 置信度: {conf:.2f}")
```

(3) YOLOv11 核心改进简介

与 YOLOv8 相比，YOLOv11 主要改进如下：

- **C3k2 模块**：改进跨阶段部分网络，在保持精度的前提下减少约 22% 参数量，更适合 CPU 推理。
- **C2PSA 注意力机制**：在颈部引入位置敏感注意力，增强对小目标（如远距离行人）的特征提取能力。
- **统一任务接口**：检测、分割、姿态估计共用同一套训练代码，切换任务只需更换模型文件。

(4) 插入截图说明

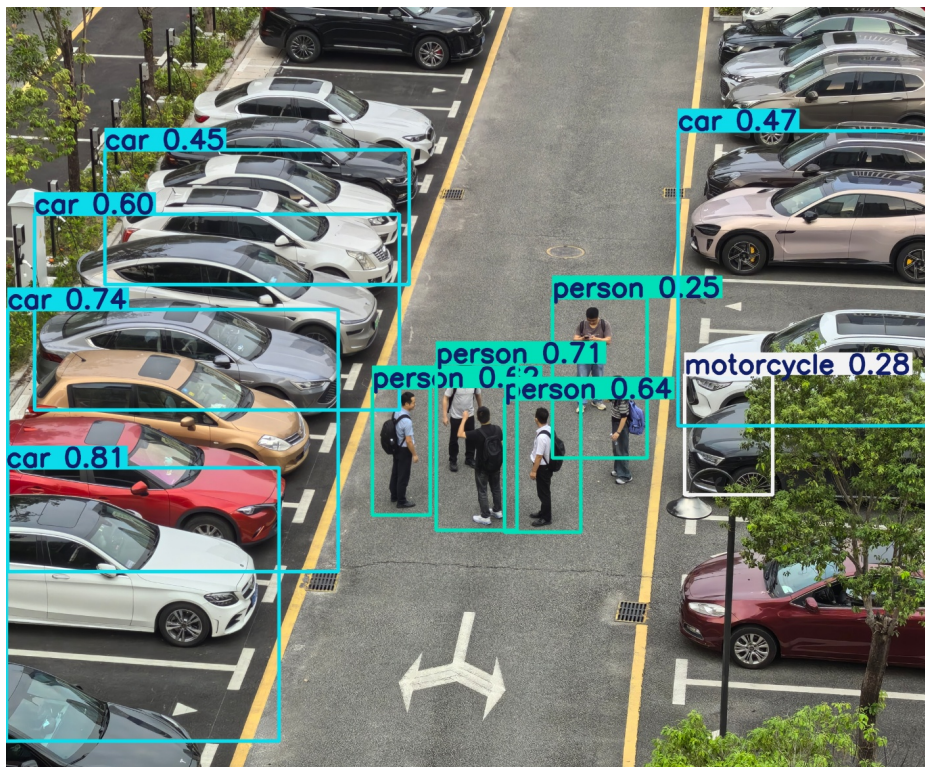


图 1 推理结果示例（蓝框为车辆，绿框为行人）

四、数据处理与分析：

以 CPU 训练 10 个 epoch 的典型结果为例：

Epoch	Box(P)	R	mAP@0.5	mAP@0.5:0.95
10	0.784	0.686	0.761	0.465

各指标含义：

- **Box(P)**：检测结果的精确率（Precision），表示模型预测为正样本的目标中，真正正确的比例。本实验中 $\text{Box(P)}=0.784$ ，说明模型检出的目标中约有 78.4% 是正确的，误检相对较少。
- **R**：召回率（Recall），表示所有真实目标中被模型成功检测出的比例。本实验中 $R=0.686$ ，说明模型能够检测出约 68.6% 的真实行人和车辆目标，仍存在一定漏检。
- **mAP@0.5**：IoU ≥ 0.5 时的平均检测精度，是目标检测最常用的评估指标。
- **mAP@0.5:0.95**：在多个 IoU 阈值下取平均，评估更严格。

由于使用 CPU 训练且 epoch 较少，最终 mAP 低于 GPU 长时间训练的结果，但已能正确检测到画面中的行人和车辆，验证了完整流程的可行性。若需提升精度，可增加 epoch 数量或使用更大规模的预训练权重 (`yolo11s.pt`)。

五、实验结论：

1. 本实验在无 GPU 的普通笔记本电脑上，通过 Roboflow 获取小型公开数据集，成功完成了 YOLOv11 的完整训练与推理流程，验证了该框架对低配硬件的良好兼容性。
2. Roboflow 提供的数据集已包含 YOLO 格式标注和 `data.yaml` 配置文件，大幅降低了数据准备门槛，无需手工标注即可快速开始训练。
3. 从实验结果来看，在训练 10 个 epoch 后，模型取得了 $\text{Box(P)}=0.784$ 、 $\text{R}=0.686$ 、 $\text{mAP@0.5}=0.761$ 、 $\text{mAP@0.5:0.95}=0.465$ 的结果，表明模型已经能够较有效地识别图像中的行人和车辆，其中精确率较高，说明误检控制较好，但召回率相对偏低，说明仍存在一定漏检现象。
4. 实验过程中，通过将 `imgsz` 从 640 降至 416、`batch` 设为 4，显著缩短了 CPU 训练时间，说明合理调整超参数对低配环境至关重要。
5. 若条件允许，后续可在云平台（如 Google Colab 免费 GPU）上进行更长时间的训练，以进一步提升检测精度。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

无

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。